

PYTHON REFERENCE GUIDE

Lists & Tuples | Dictionaries & Sets — Beginner Friendly Edition

This guide explains Python's four core data structures in plain, simple language, with a working code example and its exact output for every concept. No prior experience assumed — if you can read English, you can follow this.

1	Lists	Indexing, slicing, append, insert, extend, pop, remove, sort, comprehensions
2	Tuples	Creation, indexing, immutability, unpacking, when to use a tuple
3	Dictionaries	Keys, values, get(), items(), update(), pop(), comprehensions
4	Sets	Creation, union, intersection, difference, add/remove, membership test
5	Quick Comparison	Lists vs Tuples vs Dictionaries vs Sets — which one to use

1 LISTS

A **list** is a container that holds multiple items in order, like a shopping list. You can add, remove, or change items any time — this is called being **mutable**.

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

Output:

```
['apple', 'banana', 'cherry']
```

Indexing — getting one item

Every item has a position number called an **index**, starting from **0** (not 1). Negative numbers count backwards from the end.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
print(fruits[-1])
```

Output:

```
apple
cherry
```

In plain English: fruits[0] means 'give me the 1st item', fruits[-1] means 'give me the last item.'

Slicing — getting many items at once

Use **[start:stop:step]** to grab a chunk of the list. The 'stop' position itself is not included.

```
nums = [10, 20, 30, 40, 50]
print(nums[1:3])
print(nums[:2])
print(nums[::2])
```

Output:

```
[20, 30]
[10, 20]
[10, 30, 50]
```

append() — add one item to the end

```
fruits = ["apple", "banana"]
fruits.append("mango")
print(fruits)
```

Output:

```
['apple', 'banana', 'mango']
```

insert() — add an item at a specific position

Unlike `append()` which always adds to the end, `insert()` lets you choose exactly where.

```
fruits = ["apple", "mango"]
fruits.insert(1, "banana")
print(fruits)
```

Output:

```
['apple', 'banana', 'mango']
```

extend() — add multiple items at once

```
fruits = ["apple", "banana"]
fruits.extend(["mango", "grape"])
print(fruits)
```

Output:

```
['apple', 'banana', 'mango', 'grape']
```

Watch out: `fruits.append(['mango','grape'])` adds the *WHOLE* list as one item, giving you a nested list. Use `extend()` when you want the items added individually.

pop() — remove and return an item

Removes an item by position and gives it back to you. Without a number, it removes the **last** item.

```
fruits = ["apple", "banana", "mango"]
removed = fruits.pop()
print(removed)
print(fruits)
```

Output:

```
mango
['apple', 'banana']
```

remove() — delete an item by value

Different from `pop()` — here you give the actual value, not the position.

```
fruits = ["apple", "banana", "mango"]
fruits.remove("banana")
print(fruits)
```

Output:

```
['apple', 'mango']
```

sort()

Arranges the list in order permanently. Use **reverse=True** for descending order.

```
nums = [5, 2, 9, 1]
nums.sort()
print(nums)
nums.sort(reverse=True)
print(nums)
```

Output:

```
[1, 2, 5, 9]
[9, 5, 2, 1]
```

Watch out: `sorted(nums)` creates a *NEW* sorted list without changing the original — use this if you still need the original order later.

List Comprehensions — a shortcut to build lists

Instead of writing a loop across several lines, you can build a new list in one line.

```
squares = [x**2 for x in range(1, 6)]
print(squares)

evens = [x for x in range(10) if x % 2 == 0]
print(evens)
```

Output:

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
```

In plain English: Read it as: 'give me x squared, for every x from 1 to 5' — same idea as a for loop, just shorter.

2 TUPLES

A **tuple** looks just like a list but uses round brackets **()** instead of square brackets, and once created, it **cannot be changed** — this is called **immutable**.

Creating and Indexing

```
point = (10, 20)
colors = ("red", "green", "blue")
print(point[0])
print(colors[-1])
```

Output:

```
10
blue
```

Tuple Immutability — the key difference from lists

```
point = (10, 20)
point[0] = 99
# This line causes an error:
# TypeError: 'tuple' object does not
# support item assignment
```

Watch out: If your code needs to add/remove/change items later, use a *list*, not a *tuple*.

Tuple Unpacking — a handy shortcut

You can assign each item in a tuple to its own variable in a single line.

```
point = (10, 20)
x, y = point
print(x)
print(y)
```

Output:

```
10
20
```

Why use a tuple instead of a list?

- Use a **list** when the data needs to change — e.g. a to-do list.
- Use a **tuple** when the data should stay fixed — e.g. a date (year, month, day), or GPS coordinates.
- Tuples can be used as dictionary keys; lists cannot (you'll see why once you cover dictionaries next).
- Tuples are slightly faster than lists because Python knows they'll never change size.

3 DICTIONARIES

A **dictionary** stores data in **key : value** pairs — like a real dictionary where you look up a word (key) to find its meaning (value).

```
student = {"name": "Jeevitha", "age": 21, "course": "CS"}
print(student)
```

Output:

```
{'name': 'Jeevitha', 'age': 21, 'course': 'CS'}
```

Accessing Keys and Values

```
student = {"name": "Jeevitha", "age": 21}
print(student["name"])
print(student.keys())
print(student.values())
```

Output:

```
Jeevitha
dict_keys(['name', 'age'])
dict_values(['Jeevitha', 21])
```

get() — the safe way to look up a value

`student['grade']` crashes your program if 'grade' doesn't exist. `get()` avoids that by returning a default value instead.

```
student = {"name": "Jeevitha", "age": 21}
print(student.get("age"))
print(student.get("grade", "Not Found"))
```

Output:

21
Not Found

Watch out: Always prefer `get()` over `[]` when you're not 100% sure the key exists.

items() — looping through a dictionary

```
student = {"name": "Jeevitha", "age": 21}
for key, value in student.items():
    print(key, "-", value)
```

Output:

```
name - Jeevitha
age - 21
```

update() — add or change multiple values at once

```
student = {"name": "Jeevitha", "age": 21}
student.update({"age": 22, "city": "Chennai"})
print(student)
```

Output:

```
{'name': 'Jeevitha', 'age': 22, 'city': 'Chennai'}
```

pop() — remove a key and get its value

```
student = {"name": "Jeevitha", "age": 22}
removed = student.pop("age")
print(removed)
print(student)
```

Output:

```
22
{'name': 'Jeevitha'}
```

Dictionary Comprehensions

```
squares = {x: x**2 for x in range(1, 5)}
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

4 SETS

A **set** is a collection of items where **every item is unique** — duplicates are automatically thrown away. Sets also have no fixed order.

```
nums = {1, 2, 2, 3, 3, 3}
print(nums)
```

Output:

```
{1, 2, 3}
```

add() and remove()

```
colors = {"red", "green"}
colors.add("blue")
colors.remove("red")
print(colors)
```

Output:

```
{'green', 'blue'}
```

Watch out: Since sets have no order, the printed order may look different from the order you added items.

Membership Test — checking if something exists

The **in** keyword works on lists, tuples, and sets — but it's especially fast on sets, which is why sets are great for checking duplicates.

```
colors = {"red", "green", "blue"}
print("red" in colors)
print("yellow" in colors)
```

Output:

```
True
False
```

Set Operations — comparing two sets

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)    # union
print(a & b)    # intersection
print(a - b)    # difference
print(a ^ b)    # symmetric difference
```

Output:

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```


Symbol	Method Name	What it means
	union()	Everything from both sets, combined
&	intersection()	Only items that appear in both
-	difference()	Items in the first set, but not the second
^	symmetric_difference()	Items that are NOT shared by both

5 QUICK COMPARISON — WHICH ONE TO USE

If you forget everything else, remember this table. It answers the question every beginner asks: **'which data structure should I use?'**

Structure	Brackets	Ordered?	Changeable?	Duplicates?	Use it for...
List	[]	Yes	Yes	Allowed	A to-do list, a list of scores, anything that changes
Tuple	()	Yes	No	Allowed	Fixed data — coordinates, dates, RGB colors
Dictionary	{k:v}	Yes (3.7+)	Yes	Keys unique	Looking up info by name — student record, phonebook
Set	{ }	No	Yes	Not allowed	Removing duplicates, fast checks, comparing groups

One more thing worth knowing: len()

Works on all four structures — tells you how many items are inside.

```
print(len([1, 2, 3]))
print(len((1, 2)))
print(len({"a": 1, "b": 2}))
print(len({1, 2, 3, 3}))
```

Output:

```
3
2
2
3
```